

Efficient CPU-GPU Collaborative Inference for MoE-based LLMs on Memory-Limited Systems

En-Ming Huang*
National Taiwan University
Taipei, Taiwan
r13922078@csie.ntu.edu.tw

Li-Shang Lin*
National Tsing Hua University
Hsinchu, Taiwan
lslin@cs.nthu.edu.tw

Chun-Yi Lee†
National Taiwan University
Taipei, Taiwan
cylee@csie.ntu.edu.tw

Abstract—Large Language Models (LLMs) have achieved impressive results across various tasks, yet their high computational demands pose deployment challenges, especially on consumer-grade hardware. Mixture of Experts (MoE) models provide an efficient solution through selective activation of parameter subsets, which reduces computation requirements. Despite this efficiency, state-of-the-art MoE models still require substantial memory beyond typical consumer GPU capacities. Traditional offloading methods that transfer model weights between CPU and GPU introduce latency, limiting inference performance. This paper presents a novel CPU-GPU collaborative inference framework that incorporates an expert caching mechanism on the GPU to reduce data transfer requirements and enable faster inference through cache hits. Computations are offloaded to CPU for efficient cache miss handling, which benefits from CPU multithreading optimizations. The evaluations of our framework demonstrate performance improvements and highlight the potential of CPU-GPU collaboration to maximize hardware utilization for single-request inference scenarios on consumer-grade systems. The implementation of our framework is available at github.com/elsa-lab/MoE-CPU-GPU-Collaborative-Inference.

Index Terms—CPU-GPU optimization, LLM Inference, MoE.

I. INTRODUCTION

Large language models (LLMs) have demonstrated remarkable effectiveness in addressing diverse real-world challenges [1]–[3]. Although increased model sizes have led to improved accuracy, they have also introduced substantial computational challenges. Mixture of Experts (MoE) model architectures [4]–[10] have emerged as a computationally efficient paradigm that maintains superior performance while substantially reducing inference costs through the activation of sparse experts. For instance, Mixtral 8x7B [6] achieved performance superior to that of LLaMA2-70B [11] across various benchmarks while requiring only one-fifth [6] of the computational resources. In these architectures, the large feed-forward networks (FFNs) within each Transformer block [12] are decomposed into multiple smaller expert models. As depicted in Fig. 1, the large dense FFN is replaced by an MoE layer, wherein the router network determines which experts to activate based on the input data. This selective mechanism forms a fundamental advancement in computational efficiency.

Despite reduced computational complexity through sparsity, MoE models encounter memory constraints on consumer-

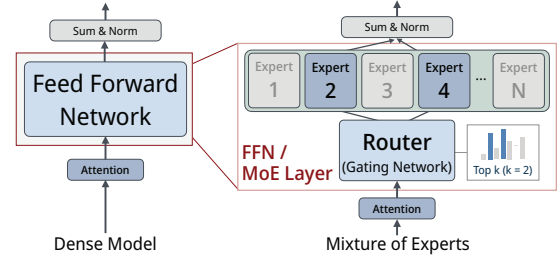


Fig. 1: Comparison of dense FFN and MoE layer architectures.

grade hardware (e.g., Mixtral 8x7B requires 80GB of model weights vs RTX 4090’s 24GB capacity). To address this limitation, researchers have proposed various offloading methods that store model weights in CPU memory or SSDs and transfer weight portions to the GPU during computation under different strategies [13]–[20]. On-demand expert fetching transfers specific experts based on router decisions, yet model weight transfers cannot proceed until router decisions are determined [13], [21]. As a result, prefetching techniques were proposed to overlap processor computations with memory transfers through expert model speculation [14]–[16]. For instance, required experts in layer- $i + 1$ are predicted in prior layers (e.g., layer- i), which allows earlier weight fetching. Alternatively, increased batch sizes have proven effective in enhancing total inference throughput through expert weight reuse [17]–[20]. However, minimum latency remains limited by the communication time needed for expert weight transfers.

To address these memory and latency challenges, this paper proposes CPU-GPU collaboration for efficient inference on consumer-grade hardware, particularly for per-request scenarios common in such systems. The primary insight stems from our observation that previous research on MoE-based LLM inference has predominantly focused on GPU-based computations, which has not fully utilized CPU computational resources and therefore remains limited by model transfer time between CPU and GPU. Despite its often overlooked potential, CPU possesses significant parallel computation capabilities supported by PyTorch [24], a widely used library that implements multithreading through OpenMP [25] and vector instructions. This parallelization is controlled by the environment variable `OMP_NUM_THREADS`, which adjusts CPU core utilization for computation. As shown in Table I and

*Authors contributed equally. † Corresponding author

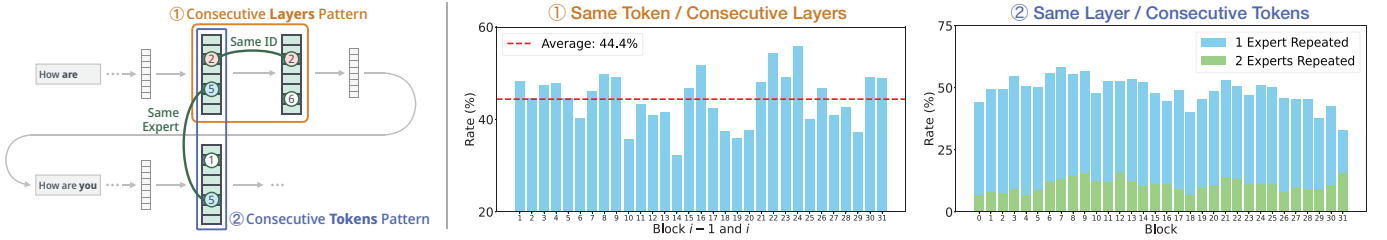


Fig. 2: Mixtral 8x7B (32 Transformer blocks) expert selection patterns on the MMLU [22], [23] dataset: (1) Consecutive Layers Pattern and (2) Consecutive Token Pattern. Each Transformer block’s router demonstrates clear patterns of expert reuse.

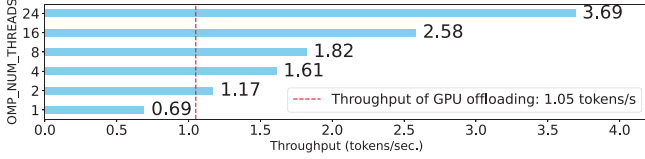


Fig. 3: Mixtral 8x7B token generation speed vs. CPU cores.

TABLE I: Communication & computing time for Mixtral 8x7B FFN using AMD 7960X CPU and NVIDIA RTX 4090 GPU with PCIe 4.0 $\times 16$ bandwidth (64 GB/s bidirectional).

Device	GPU	CPU (w/ different no. of threads)					
OMP_NUM_THREADS	-	1	2	4	8	16	24
Computing time (ms.)	0.3	44.1	25.5	18.3	15.8	11.0	7.3
Communication time (ms.)	28.0				0.1		

Fig. 3, our analysis reveals that CPU computation can achieve superior efficiency compared to GPU offloading for certain operations in the Mixtral 8x7B model. This advantage stems from the interplay between computation and communication costs. Although the GPU exhibits substantially faster computational speed (0.3 ms versus 25.5 ms with two threads), the significant communication overhead (28.0 ms) negates this benefit in per-request scenarios. The throughput analysis in Fig. 3 further suggests that CPU configurations with more than two threads offer superior throughput as compared to GPU offloading.

To demonstrate that expert computations can be effectively conducted on CPU rather than through GPU weight transfers, Table I compares transfer times and CPU computation times on an AMD Ryzen Threadripper 7960X CPU and NVIDIA GeForce RTX 4090 GPU. The communication time in the CPU version reflects output transfer from the GPU-computed attention block back to CPU. The results show that two CPU cores can surpass GPU offloading performance for the Mixtral 8x7B model. Fig. 3 illustrates overall token generation speed under different CPU core configurations. The results further indicate that employing at least two CPU cores achieves greater efficiency than conventional MoE layer GPU offloading. These profiled findings reveal that prefetching methods [14]–[16] cannot achieve effective computation-communication overlap, as MoE expert transfer costs substantially exceed computation time. As a result, prefetching performance improvements remain constrained by inherent communication latency.

Based on CPU computational capabilities and experimental insights, we introduce a CPU-GPU collaborative inference

framework for MoE-based LLMs. This framework incorporates GPU expert caching and introduces novel asynchronous cache miss handling to minimize latency penalties. When required experts are not cached on GPU, computations are offloaded to CPU while experts are asynchronously fetched to GPU for subsequent token generation. Since GPU cannot accommodate all MoE layer experts, we consider GPU memory as a cache for expert subsets. We observe expert reuse patterns across consecutive token generations in MoE models. Real-case study evidence on Mixtral 8x7B model appears in Fig. 2, which shows selected experts are re-employed in subsequent token generation with 30% to 50% probability. Performance comparisons in Table I and Fig. 3 further demonstrate that employing CPU to complement non-cached layer computations offers an effective solution. This methodology enables synergistic CPU-GPU resource utilization that achieves throughput surpassing GPU-only computation. Unlike prior works requiring model alteration or dataset profiling [14], [15], [26], ours allows out-of-the-box operation with any MoEs.

We evaluate the framework performance on state-of-the-art (SoTA) MoE-based LLM models: Mixtral 8x7B [6] and Phi-3.5-MoE [7] using an AMD Ryzen Threadripper 7960X CPU and NVIDIA GeForce RTX 4090 GPU. Our solution surpasses all previous methods [13], [15], [16], [21], [26] without accuracy trade-offs, including approaches that required model alterations [15] and dataset profiling [26]. Specifically, our approach achieves up to 4.8 and 10.4 tokens per second for Mixtral 8x7B and Phi-3.5-MoE respectively in single-request scenarios, demonstrating a 4.4 $\times$ speedup against the prior prefetching-based approach [15]. The main contributions are:

- 1) A CPU-GPU collaborative inference framework for MoE-based LLMs on resource-constrained consumer-grade systems that addresses single-request scenarios common in real-world cases with limited batch sizes. Comprehensive experiments on state-of-the-art MoE models validate our framework’s effectiveness without requiring model alterations or additional prior profiling.
- 2) The demonstration of the untapped potential of CPU computational resources through multi-threading capabilities that surpass conventional GPU-only methods.
- 3) An innovative *asynchronous expert caching mechanism* that exploits data locality in GPU memory and leverages CPU multi-core resources during cache misses while enabling asynchronous expert weight transfers to GPU.

II. RELATED WORK AND MOTIVATIONAL INSIGHTS

A. Related Work

To address deployment challenges of large MoE-based LLMs on memory-limited GPUs, existing offloading strategies exhibit fundamental limitations across distinct approaches. Deepspeed Inference [13] and Huggingface Accelerate [21] provide on-demand expert weight fetching based on router decisions without requiring model modifications, but lack prefetching mechanisms that create significant communication latency. Throughput-oriented methods including ExpertFlow [17] and MoE-Infinity [18] enhance multi-user performance through token scheduling, expert prefetching, and quantization but prioritize overall system throughput over individual request latency. Edge-focused approaches such as EdgeMoE [27] and SwapMoE [28] target severely constrained devices through extensive model modifications and offline profiling for quantization and pruning strategies. Router modification strategies exemplified by Pre-gated MoE [15] and AdapMoE [16] pre-select experts for subsequent layers based on current activations. These methods require custom prefetch layers and router logic modifications that reduce compatibility with standard inference frameworks. Router mispredictions can stall GPU execution while awaiting weight transfers, introducing substantial communication overhead. Heterogeneous system approaches such as Fiddler [26] utilize both CPU and GPU resources through offline profiling to cache frequently activated experts based on skewed activation distributions. This profiling-dependent methodology limits generalization across MoE architectures and employs greedy algorithms with computational overhead of $O(N \times 2^N)$, creating inefficiency for larger models. Our methodology addresses these limitations by requiring no model architecture modifications, quantization, or extensive offline profiling while supporting diverse MoE-based LLM models with maintained compatibility and accuracy.

B. Motivational Observation: Expert Selection Pattern

Identifying experts that are frequently reused across model layers and consecutive tokens allows us to cache and prefetch portions of these expert weights on the GPU, which enables faster access and improves efficiency. Fig. 2 illustrates two patterns: *Consecutive Layers Pattern* and *Consecutive Tokens Pattern*. To explore this potential influence of them, we recorded expert selection results from each layer’s router in the Mixtral 8x7B model and analyzed these two patterns.

① **Consecutive Layers Pattern:** During each token generation pass, consecutive layers frequently route to identical expert sets. Fig. 2 reveals that approximately 44% of the time, routers direct experts to the same ID as in the previous block. This consistency enables expert selection prediction for weight prefetching. Mixtral-offloading [14] leverages this pattern through subsequent router networks to predict future expert activations. However, this strategy proves inadequate for offloading scenarios since required expert weights must load within the brief time window of a single layer’s forward pass. Similarly, DeepSpeed Inference and Huggingface Accelerate

attempt to hide communication latency through sequential-layer preloading but fail to exploit token-level expert reuse across layers, limiting their ability to benefit from repeated expert activations. We identify reuse patterns not only across layers but also across tokens, which we term the *Consecutive Tokens Pattern*. This newly observed behavior enables more effective expert weight prefetching through our caching strategy design in memory-constrained offloading scenarios.

② **Consecutive Tokens Pattern:** Our observations indicate that within the same layer across consecutive token generations, identical experts are frequently selected. Fig. 2 illustrates the frequency of consecutive token generations where identical experts are routed within a given layer. Results demonstrate that at least one expert maintains reuse probability between 40% and 60% across all layers. This pattern suggests performance benefits through GPU-based expert caching, as experts selected for current generation likely serve subsequent tokens. Unlike the *Consecutive Layers Pattern*, which demands caching within a single layer’s brief pass, the *Consecutive Tokens Pattern* offers an extended window for expert weight fetching during the current token’s forward pass, enabling greater computation-communication overlap. Further analysis reveals that within each block, the same expert tends to remain active beyond the immediate next token. Among tokens where at least one expert repeats from the previous token, approximately 23% share at least one expert with the previous two tokens, and 18% share with the previous three or more tokens. These patterns indicate high probability that activated experts continue for subsequent tokens. Rather than relying on offline profiling of static activation distributions, our approach dynamically observes expert reuse patterns and adapts accordingly, enabling applicability across different MoEs. We implement a Least Recently Used (LRU)-style caching strategy guided by the *Consecutive Tokens Pattern*, which reflects the observation that recently selected experts are likely to be reused shortly.

III. METHODOLOGY

A. Framework Overview

Our MoE inference framework is developed for optimizing memory usage and computational efficiency by selectively caching expert weights on the GPU. Its essential model components, such as self-attention layers, router networks, and KV caches, reside continuously in the GPU’s memory. Given that MoE expert weights constitute the dominant portion of model parameters and memory requirements, the framework allocates the remaining GPU memory to implement an expert cache for storing expert weights. This cache employs an N -index, M -way set-associative structure spanning layers 0 through $N - 1$ in the MoE model. Each layer maintains capacity for M expert weights to enable rapid access and minimize data transfer needs. Fig. 4 (a) illustrates the workflow, with key steps described as follows: ① **Cache Check:** Upon each layer access, the framework checks whether the required experts are available in GPU’s expert cache. ② **Execution Based on Cache Status:** If an expert resides in the expert cache (i.e., a cache hit), the computation proceeds directly

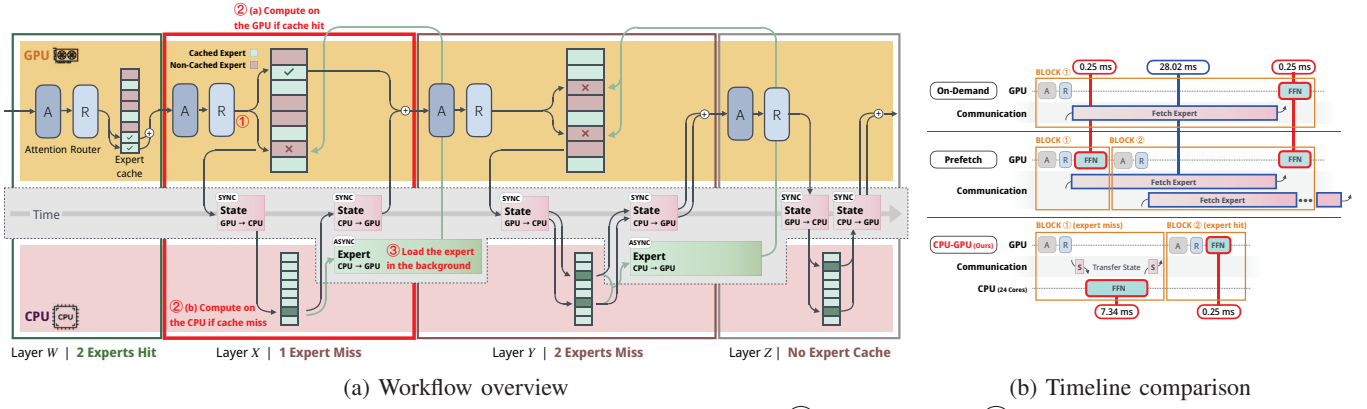


Fig. 4: (a) Our workflow, and (b) timing comparison. Workflow includes ① cache checks, ② execution based on cache status (②(a) on GPU if cache hit, ②(b) on CPU if cache miss), and ③ asynchronous data transfer to update the GPU cache for future token generation. For layers beyond the cache coverage (e.g., Layer Z), all expert computations are done on the CPU.

on the GPU so as to minimize communication overhead. For instance, in layer W , where all necessary experts are cached, there is no data transfer between CPU and GPU. In contrast, when a cache miss occurs (e.g., in layers X and Y), our framework offloads the intermediate state (i.e., the output from the attention layer) to the CPU side for expert computation. ③ **Asynchronous Data Transfer:** For missed experts, the required weights are asynchronously copied to the GPU in the background (i.e., post-fetching) to update the cache for future access. In the case of cache-missed layers, CPU resources are temporarily leveraged for expert computation while the missing weights are transferred back to the GPU. In addition, for layers beyond the cache coverage (e.g., layer Z , where cache use is unavailable due to the insufficient size of available GPU memory, all expert computations are conducted on CPU.

Fig. 4 (b) depicts the timeline comparison of our method against on-demand fetching and prefetching methodologies (e.g., Pre-gated MoE [15] and AdapMoE [16]). Prefetching method efficiency remains limited since computation time is significantly less than communication cost. Although visualization suggests overlapping expert transfers in Pre-gated MoE, their actual implementation of data transfers remains serialized. In contrast, our collaborative framework reduces data transfer by offloading expert computations to CPU, enabling accelerated token generation. Meanwhile, the expert cache and asynchronous transfers further ensure efficient GPU utilization as data transfers can be overlapped with computations.

B. Memory Management and Cache Design

Our framework divides GPU memory into two sections: an expert cache and a section for other essential model components, including weights for self-attention, normalization, router networks, and the KV cache. These components typically require a small amount of memory (e.g., Mixtral 8x7B requires approximately 5 GB for them), with remaining GPU memory allocated to the expert cache. Expert cache configuration depends on available GPU memory and individual

expert size. The number of expert slots (S) is determined by dividing the available GPU memory by the size of an individual expert as $S = \left\lfloor \frac{\text{available GPU memory}}{\text{per-expert size}} \right\rfloor$. The number of expert slots represents the total cache capacity in terms of expert count. Through specification of the number of ways per layer M , the number of indexes (N) in the cache is calculated as: $N = \left\lfloor \frac{S}{M} \right\rfloor$. For example, on an NVIDIA GeForce RTX 4090 with 24 GB of memory, around 56 expert slots can be accommodated for the Mixtral 8x7B model, which consists of 256 experts in total (i.e., eight experts per layer across 32 layers). Configuring the GPU memory as a 4-way set-associative cache results in 14 indexes (i.e., $\frac{56}{4} = 14$). Such a configuration enables expert coverage for layers 0 to 13.

Our expert cache employs an LRU eviction policy. Due to the *Consecutive Tokens Pattern* property discussed in Section II-B, this LRU policy ensures that frequently accessed experts remain in the cache. Our experiments also evaluated the LRU policy against other strategies, including First In, First Out (FIFO) and random selection, all in Section IV-D.

C. Efficient Transfer & Implementation Details

To enhance data transfer performance, our framework leverages NVIDIA’s dual-copy engine capability, which enables concurrent data transfers. This parallel transfer capability is implemented through two independent CUDA streams. Without these dual streams, data transfers would be serialized, creating a performance bottleneck, as transfers of intermediate states (i.e., attention outputs) from GPU to CPU could need to wait for ongoing expert weight transfers to complete.

Our CPU-GPU collaborative framework builds upon the official MistralAI repository [29], which implements the Mixtral 8x7B models. We furthermore adapted this codebase to support Hugging Face-formatted models (e.g., Phi3.5-MoE).

IV. EXPERIMENTAL RESULTS AND INSIGHTS DISCUSSION

A. Experimental Setup

We compare our proposed framework against on-demand expert fetching [13], [21] expert prefetching framework, Pre-gated MoE [15], and a CPU-GPU collaboration framework,

They created a single CUDA stream for the fetcher (i.e., the structure that fetches required data from CPU to GPU) in the released source code, so all communications on the same CUDA stream are performed sequentially.

TABLE II: Configuration comparison of two MoE-based LLM models: Mixtral 8x7B and Phi3.5-MoE.

Model	Mixtral 8x7B [6]	Phi3.5-MoE [7]
# of parameters	46.7B	41.9B
Model size	88 GB	79 GB
# of layers	32	32
# of experts per layer	8	16
Select top-K experts	2	2
Size per expert	340 MB	152 MB

TABLE III: Expert computation and communication time (in milliseconds) for the Mixtral 8x7B and Phi3.5-MoE models.

Model	Device CPU threads	GPU -	CPU (w/ different no. of threads)					
			1	2	4	8	16	24
Mixtral 8x7B	Expert comp. time	0.25	44.12	25.53	18.34	15.76	10.96	7.34
	Comm. time	28.02			0.11			
Phi3.5-MoE	Expert comp. time	0.11	22.73	12.80	8.58	6.39	3.92	3.36
	Comm. time	12.26			0.11			

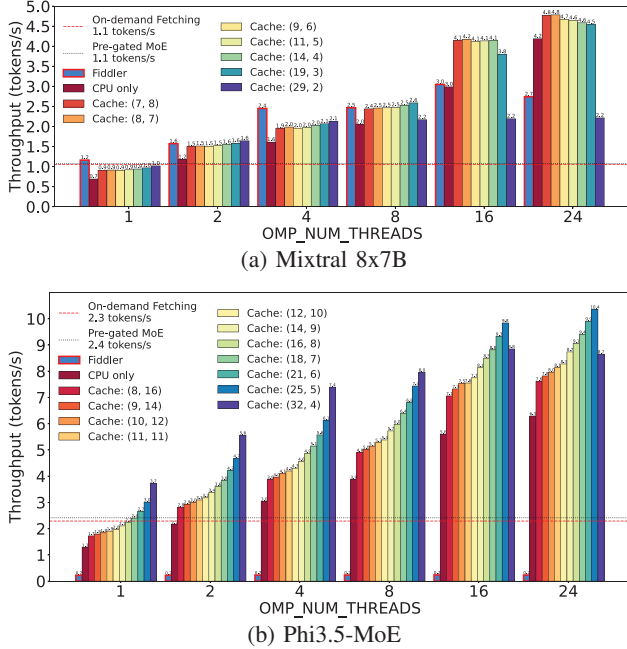


Fig. 5: Overall performance comparison of (a) Mixtral 8x7B and (b) Phi3.5-MoE models under different number of CPU cores (OMP_NUM_THREADS) and cache configurations. The cache legend indicates (# of indexes, # of ways).

Fiddler [26], on two state-of-the-art MoE-based LLMs: Mixtral 8x7B [6] and Phi3.5-MoE [7]. Since Pre-gated MoE implementation does not support these newer models, we estimate its performance by assuming perfect computation-communication overlap. Table II compares the Mixtral 8x7B and Phi3.5-MoE models. Both models select two experts per layer for inference; however, Phi3.5-MoE employs smaller expert sizes that result in reduced activated parameters. The expert cache capacity in slot count is determined by available GPU memory and individual expert size, according to the details in Section III-B. Our experiments are conducted on a system equipped with an AMD Ryzen Threadripper 7960X 24-core CPU and NVIDIA RTX 4090 24GB GPU. The maximum CPU frequency varies between 4.8GHz and 5.3GHz based on active core count, where higher core usage reduces frequency. The GPU connects to the CPU via PCIe Gen 4.0 $\times 16$ interconnect. Our power analysis monitors CPU package power through Running Average Power Limit (RAPL) and GPU power through the `nvidia-smi` command.

B. Overall Performance Analysis

Fig. 5 presents a comprehensive performance comparison of our proposed CPU-GPU collaborative inference framework with on-demand expert fetching [13], [21], the prefetching strategy (Pre-gated MoE) [15] and Fiddler [26] under varying numbers of CPU cores. The *CPU only* configuration refers to the scenario where no GPU expert cache is employed, and all expert computations occur on the CPU. With two to four CPU cores, the CPU-only version surpasses the prefetching works, as the reduced CPU computation time becomes shorter than the PCIe transfer time for expert weights. This configuration, where all computations are performed on the CPU, establishes the performance lower bound for our method, corresponding to a scenario with a 100% cache miss rate. Furthermore, with the employment of the GPU expert cache, our framework achieves approximately 15% ~ 35% and 50% ~ 250% performance improvements over the *CPU only* setup for Mixtral 8x7B and Phi3.5-MoE, respectively.

Our framework significantly outperforms existing methodologies, achieving up to $4.4\times$ speedup for Mixtral 8x7B and $4.3\times$ for Phi3.5-MoE with full 24-core CPU utilization, while surpassing Fiddler by approximately $1.6\times$. Note that Fiddler performs poorly on Phi3.5-MoE due to its reliance on offline profiling of expert popularity, which limits applicability to other MoE models. Furthermore, the increased expert count in Phi3.5-MoE introduces significant overhead for Fiddler’s CPU-GPU execution decisions due to exponential complexity.

Our collaborative CPU-GPU framework significantly boosts inference throughput by efficiently coordinating resources. Even with minimal CPU cores, like a single one, our method outperforms Pre-gated MoE by 54% on the Phi3.5-MoE model. With just two CPU cores, it consistently surpasses previous methods on both tested models, showcasing its efficiency across diverse resource configurations. This flexibility means our framework is not limited to high-core systems and can adapt to various hardware setups. Furthermore, our approach does not require any model modifications and dataset profiling, ensuring compatibility with new MoE models. While Pre-gated MoE necessitates router network changes, complicating reproducibility and accuracy evaluations, Fiddler requires profiling on dataset to select which experts are placed on the GPU. Additionally, Fiddler’s algorithm for determining an expert model’s execution unit (e.g., one the CPU or the GPU) can serve as a bottleneck for models with a large number of experts due to its exponential complexity. We offer a straightforward integration and robust solution for enhancing inference performance of MoE model variants.

C. Performance vs CPU Count and Cache Configurations

Table III provides a detailed breakdown of the expert’s computation time and communication time for the Mixtral 8x7B and Phi3.5-MoE models across different CPU core counts. These evaluation results, along with Fig. 5, reveal two primary performance trends as follows:

With Fewer CPU Cores (1~4): In low-core configurations, expert computation time per core is significantly higher than communication time. For example, with a single CPU core, the computation time for an expert layer in Mixtral 8x7B reaches 44.12 ms, whereas the computation time on the GPU is only 0.25 ms. This imbalance highlights expert’s computation as the primary bottleneck. As a result, cache configurations with a higher number of indexes and fewer ways are optimal in this scenario. By maximizing expert reuse on the GPU and covering more layers, this setup achieves greater speedup to effectively offset the high computation cost on the CPU. Nevertheless, the trade-off of using fewer cache ways results in an increased cache miss rate, which leads to more frequent CPU-GPU transfers. This trade-off is favorable in low-core scenarios since the computation dominates the total latency to make the added communication overhead less impactful. However, this configuration becomes less advantageous as the number of CPU cores increases.

With Increased CPU Cores (8~24): As the number of CPU cores increases, expert’s computation time decreases significantly due to parallelism. For instance, the computing time for Mixtral 8x7B drops from 44.1 ms on one core to 7.3 ms on 24 cores. Under these high-core conditions, communication overhead in low-way cache configurations becomes the primary bottleneck, as the reduced computation time cannot fully mask data transfer latency. In the Mixtral 8x7B model, for example, a low-way configuration like (29,2) caps performance at around 2.2 tokens/second to indicate that data transfer rather than computation is limiting further gains. Therefore, cache configurations with more ways and fewer indexes are more efficient. This structure improves cache hit rates and reduces the frequency of CPU-GPU transfers, minimizing data transfer overhead. These trends highlight the importance of cache adjustments to balance CPU utilization and transfer efficiency as cores scale up.

D. Cache Eviction Policy Analysis

Fig. 6 illustrates the cache hit rate under various cache configurations and eviction policies for Mixtral 8x7B and Phi3.5-MoE. Two types of hit rates are presented: “expert(s) hit”, indicating at least one expert is present in the cache, and “2 experts hit”, indicating both experts selected by the router network are present. The random policy involves randomly selecting a set of expert networks to be stored in the cache statically. Since these experts remain fixed in the cache, the hit rates can be calculated using the following equations: $P(\text{At least 1 expert hit}) = 1 - \left(\frac{n-M}{n}\right) \times \left(\frac{n-M-1}{n-1}\right)$, $P(2 \text{ experts hit}) = \left(\frac{M}{n}\right) \times \left(\frac{M-1}{n-1}\right)$, where M represents the number of cache ways for each index, and n

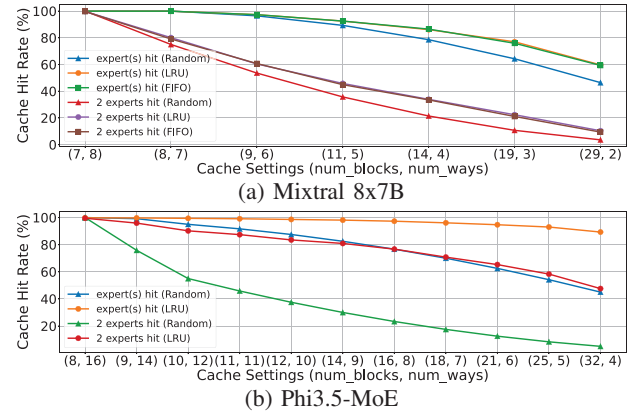


Fig. 6: Expert cache hit rate comparisons for two MoE-based LLM models: (a) Mixtral 8x7B and (b) Phi3.5-MoE under different expert cache configurations and eviction policies.

denotes the number of experts per layer. These are labeled as “expert(s) hit (Random)” and “2 experts hit (Random)” in Fig. 6, respectively. In Fig. 6a, the experimental results indicate that both LRU and FIFO policies perform similarly, with LRU slightly outperforming FIFO. As a result, Fig. 6 (b) only incorporates a comparison between LRU and the random policy to avoid potential misleading interpretations.

The results in Fig. 6 demonstrate that the LRU replacement policy in the expert cache outperforms the random policy. For Mixtral 8x7B, LRU improves the probability by around 5 ~ 15%, while for Phi3.5-MoE, LRU produces significantly better results than random selection. These improvements stem from two main factors: (1) Although Phi3.5-MoE contains twice the number of experts, this also increases the number of ways, which enables the expert cache to accommodate more unique experts. (2) The expert selection tends to remain stable between consecutive forward passes (e.g., generating two tokens). For example, if experts 0 and 1 were chosen in the previous pass, it is probable that one of them would be reused in the next pass, along with a new expert. The increased number of ways also makes it more likely that this new expert would be in the cache. This analysis of our cache eviction policy highlights the effectiveness of our expert-fetching mechanism, even without immediate fetching for each forward pass, as seen in methods such as [15] and other prefetching-based techniques [14], [16], [17]. By leveraging the CPU for expert computations alongside an proper cache eviction policy, our framework effectively balances expert reuse with cache management to achieve improved overall inference performance.

E. Power and Energy Analysis

Table IV presents the power consumption of our collaborative approach across different CPU core counts, compared to the prefetching method [15]. For each CPU core configuration, we select the optimal cache setup based on findings in Section IV-B. In our method, CPU power consumption is generally higher than in previous methods due to increased

TABLE IV: Power (W) comparison of our method with different numbers of CPUs against Pre-gated MoE [15].

Model	Device	(Ours) OMP_NUM_THREADS						Prefetching [15]
		1	2	4	8	16	24	
Mixtral 8x7B	CPU	86.1	91.7	100.3	111.0	133.4	147.5	92.1
	GPU	91.6	92.8	101.0	103.4	99.6	97.9	96.3
Phi3.5-MoE	CPU	84.4	88.4	92.0	98.4	110.1	118.3	88.2
	GPU	97.4	100.7	105.9	109.2	106.0	109.2	100.7

TABLE V: Joule-per-token comparison of our method with different CPU cores against Pre-gated MoE [15].

Model	(Ours) OMP_NUM_THREADS						Prefetching [15]
	1	2	4	8	16	24	
Mixtral 8x7B	177.7	115.3	95.8	82.5	55.5	51.1	171.3
Phi3.5-MoE	49.1	33.8	26.7	25.9	22.1	21.9	78.7

CPU core utilization. On average, each core consumes approximately 2 ~ 4 Watts, as measured by RAPL. Computing expert networks directly on the CPU and leveraging expert caching reduces the frequency of CPU-GPU communication and higher power consumption, indicating that our framework effectively maximizes available resources.

Regarding overall energy consumption, our reduced generation time enables a substantial reduction. Although increased CPU utilization leads to higher power draw, it is significantly more energy-efficient than prefetching methods. Table V illustrates the energy required for generating a token, showing that our method demonstrates significant energy savings across all CPU core configurations. For instance, with 24 CPU cores, our method consumes only 29.9% of the energy required by the prefetching method for the Mixtral 8x7B model and 27.8% for the Phi3.5-MoE model. The framework’s ability to adapt to different CPU core counts ensures that it remains energy-efficient across various hardware configurations.

V. CONCLUSION

We introduced a novel CPU-GPU collaborative inference framework for MoE-based language models on memory-limited GPU systems. Our approach efficiently leverages CPU multi-core parallelism for expert computations and utilizes GPU memory as an expert cache with an asynchronous fetching policy, outperforming prefetching methods by overlapping computations and communication. Validated on Mixtral 8x7B and Phi-3.5-MoE, our framework achieved up to 4.8 and 10.4 tokens/second, respectively, demonstrating significant speedups (up to 4.4×) and improved power efficiency.

VI. ACKNOWLEDGEMENTS

The authors gratefully acknowledge the support from the National Science and Technology Council (NSTC) in Taiwan under grant numbers NSTC 114-2221-E-002-069-MY3, NSTC 113-2221-E-002-212-MY3, NSTC 114-2218-E-A49-026, and NSTC 114-2640-E-002-006. The authors would also like to express their appreciation for the support from Google Inc. Furthermore, the authors extend their gratitude to the National Center for High-Performance Computing for providing the necessary computational and storage resources.

REFERENCES

- [1] T. Brown, B. Mann *et al.*, “Language models are few-shot learners,” in *Proc. Advances in Neural Information Processing Systems (NIPS)*, vol. 33, 2020, pp. 1877–1901.
- [2] OpenAI, J. Achiam *et al.*, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [3] A. Dubey, A. Jauhri *et al.*, “The llama 3 herd of models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [4] N. Shazeer, A. Mirhoseini *et al.*, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” 2017. [Online]. Available: <https://arxiv.org/abs/1701.06538>
- [5] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” 2022. [Online]. Available: <https://arxiv.org/abs/2101.03961>
- [6] A. Q. Jiang, A. Sablayrolles *et al.*, “Mixtral of experts,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.04088>
- [7] M. Abidin, J. Aneja *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [8] T. M. R. Team, “Introducing DBRX: A new state-of-the-art open llm,” March 2024, accessed: 2024-10-01. [Online]. Available: <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>
- [9] A. Yang, B. Yang *et al.*, “Qwen2 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.10671>
- [10] M. AI, “Cheaper, better, faster, stronger,” 2024, accessed: 2025-04-01. [Online]. Available: <https://mistral.ai/news/mixtral-8x22b>
- [11] H. Touvron, L. Martin *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [12] A. Vaswani, N. Shazeer *et al.*, “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems (NIPS)*, 2017, p. 6000–6010.
- [13] R. Y. Aminabadi, S. Rajbhandari *et al.*, “Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale,” in *Prof. Int. Conf. for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–15.
- [14] A. Eliseev and D. Mazur, “Fast inference of mixture-of-experts language models with offloading,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.17238>
- [15] R. Hwang, J. Wei *et al.*, “Pre-gated MoE: An algorithm-system co-design for fast and scalable Mixture-of-Expert inference,” in *Proc. Int. Symp. on Computer Architecture (ISCA)*, 2024, pp. 1018–1031.
- [16] S. Zhong, L. Liang, Y. Wang, R. Wang, R. Huang, and M. Li, “Adapmoe: Adaptive sensitivity-based expert gating and management for efficient moe inference,” in *Proc. Int. Conf. on Computer-Aided Design (ICCAD)*, 2024.
- [17] X. He, S. Zhang *et al.*, “Expertflow: Optimized expert activation and token allocation for efficient mixture-of-experts inference,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.17954>
- [18] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, “MoE-Infinity: Offloading-efficient MoE model serving,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.14361>
- [19] Y. Sheng, L. Zheng *et al.*, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2023, pp. 31 094–31 116.
- [20] W. Lee, J. Lee, J. Seo, and J. Sim, “InfiniGen: Efficient generative inference of large language models with dynamic KV cache management,” in *Proc. USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 155–172.
- [21] H. Face, “Transformers,” 2024, accessed: 2024-10-01. [Online]. Available: <https://github.com/huggingface/transformers>
- [22] D. Hendrycks, C. Burns *et al.*, “Measuring massive multitask language understanding,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2021.
- [23] —, “Aligning AI with shared human values,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2021.
- [24] A. Paszke, S. Gross *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>
- [25] L. Dagum and R. Menon, “OpenMP: an industry standard api for shared-memory programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.

- [26] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, “Fiddler: CPU-GPU orchestration for fast inference of Mixture-of-Experts models,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025.
- [27] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, “EdgeMoE: Fast on-device inference of MoE-based large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2308.14352>
- [28] R. Kong, Y. Li *et al.*, “SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget,” in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [29] mistralai, “mistral-inference,” 2024, accessed: 2025-02-14. [Online]. Available: <https://github.com/mistralai/mistral-inference>